Secure Hybrid SaaS Integration for OPERA PMS

Solution Architecture Options — Revised

Document Version: 2.0
Status: Draft
Reference PRD Version: 1.0 (Amended)
Supersedes: Version 1.0

Changelog v1.0 → v2.0

Constraint	v1.0	v2.0
TR-1	Pull-based sync only; no cloud-initiated push	~~Removed~~ — bidirectional sync now permitted
TR-2	Sync interval ≥ 1 hour or async OXI/OHIP events	**Replaced** — OPERA must remain in continuous real-time sync with the MERN cloud service
TR-3	No static IP or port forwarding at hotel site	**Relaxed** — static IP is now permitted where operationally justified
SC-3	Integration DB user restricted to `SELECT` only	**Expanded** — `INSERT` and `UPDATE` are now required for bookings and reservations at minimum
Write scope	SaaS sidecar/OHIP only	**Expanded** — writes to OPERA reservation and booking tables are a core requirement

---

Table of Contents

1. Updated Constraints Summary
2. Hybrid Read/Write Solutions (5 Options)
3. Comparison Matrix
4. Recommended Architecture

---

1. Updated Constraints Summary

Constraint	Requirement
TR-1	Bidirectional sync permitted; both push and pull patterns are acceptable
TR-2	OPERA must remain in **continuous, real-time sync** with the MERN cloud service — eventual consistency is not acceptable for bookings and reservations
TR-3	Static IP and port forwarding are permitted at the hotel site where operationally justified
SC-1	TLS 1.3 for all data in transit — non-negotiable
SC-2	PII masked/hashed before leaving on-premises unless required for core booking function

Constraint	Requirement
SC-3	Integration DB user must support `SELECT`, `INSERT`, and `UPDATE` on reservation and booking tables at minimum; `DELETE` remains prohibited
SA-1	MFA required for all external users
SA-2	General stakeholder reads target Cloud Read-Replica; booking write operations use a dedicated write path

> **Important note on SC-2 and bookings:** Guest name, contact details, and identification data are required for reservation operations. These fields are exempt from PII masking on the write path but must still be encrypted in transit (SC-1) and at rest in the cloud database.

> **Important note on SC-3:** A dedicated DB user with scoped `INSERT`/`UPDATE` permissions on `RESERVATION`, `RESERVATION_NAME`, `ALLOTMENT`, and related tables must be provisioned separately from the read-only reporting user. `DELETE`, `DROP`, `TRUNCATE`, and DDL operations remain strictly prohibited.

---

2. Hybrid Read/Write Solutions

Solution 1: OHIP REST API (Bidirectional) + MERN Backend + MongoDB Atlas Sync

Architecture Summary:
The most Oracle-aligned option. The MERN cloud service communicates with OPERA exclusively through **Oracle's OHIP REST API** — the officially certified integration layer for OPERA Cloud. The MERN Node.js backend calls OHIP endpoints to read availability and reservation data, and to write new bookings directly into OPERA. A **MongoDB Change Stream** on Atlas detects reservation mutations and triggers outbound OHIP write calls in near real-time, keeping both systems continuously in sync per TR-2. A static IP on the hotel's network is used to whitelist OHIP inbound traffic, satisfying the relaxed TR-3.

```
[MERN React Frontend]
        │ REST
        ▼
[Node.js Express Backend]
        │ OHIP OAuth 2.0 token
        ├──────────────────────────────────────────────┐
        │ Read: GET /reservations, /availability        │ Write: POST /reservations
        ▼                                               ▼
[OHIP REST API — Oracle Managed]──────────────────────►[OHIP REST API]
        │                                               │
        ▼                                               ▼
[OPERA Oracle DB — Read via OHIP]          [OPERA Oracle DB — Write via OHIP]
        │
        ▼
[MongoDB Atlas — Read Replica / Cache]
        │ Change Streams
        ▼
[Stakeholder Dashboard (SA-2)]
```

Sync Mechanism: MongoDB Atlas acts as a read cache populated by OHIP polling or webhooks. Write operations bypass MongoDB entirely and go direct OHIP → OPERA to ensure transactional consistency. OHIP webhooks notify

the MERN backend of OPERA-side changes (e.g., walk-ins, PMS-originated modifications) to keep Atlas in sync.

Compliance Mapping:

- ☑ TR-1: Bidirectional via OHIP API; both read and write supported natively
- ☑ TR-2: OHIP webhooks deliver OPERA state changes to MERN in near real-time; write confirmations are synchronous
- ☑ TR-3: Static IP whitelisted in OHIP configuration; no raw port forwarding to Oracle DB
- ☑ SC-1: OHIP enforces TLS 1.3; Atlas TLS 1.3 in transit
- ☑ SC-2: PII transmitted only for reservation operations where required
- ☑ SC-3: No direct Oracle DB user — all access mediated by OHIP, which enforces its own permission model
- ☑ SA-1/SA-2: MFA at MERN auth layer; stakeholder views use Atlas replica

Tech Stack: OHIP REST API, Node.js/Express, MongoDB Atlas, React, Atlas Change Streams
Complexity: ★★★ Medium
Cost Profile: Medium-High (OHIP subscription + Atlas M10+ cluster)

Pros:

- Writes to OPERA are validated by Oracle's own business logic layer — OHIP will reject invalid reservation states, double-bookings, or rate plan violations before they reach the DB
- No direct Oracle DB credentials required in the MERN codebase; the attack surface is dramatically reduced
- OHIP webhooks provide genuine push notification from OPERA to the MERN backend, satisfying TR-2's continuous sync requirement without polling overhead
- The cleanest long-term maintainability story — as Oracle evolves OPERA Cloud, OHIP absorbs the schema changes and the MERN backend is insulated
- MongoDB Atlas Change Streams give the React frontend a real-time data layer via websockets with minimal backend code

Cons:

- Hard dependency on OHIP subscription and Oracle licensing — the most expensive solution in this set, and Oracle's enterprise licensing process can take weeks
- OHIP is only available for OPERA Cloud deployments; on-premises OPERA 5.x installations require a separate OPERA Web Services (OWS) approach
- OHIP's published write endpoints may not cover every booking edge case — complex group allotments, packages, and multi-room reservations may require workarounds
- The MERN backend must implement OHIP token refresh, retry logic, and webhook signature verification, adding non-trivial integration boilerplate
- Real-time sync depends on OHIP webhook reliability; a webhook outage means the Atlas replica drifts until the next poll cycle

---

Solution 2: Direct Oracle DB Connection via Encrypted Tunnel + MERN ORM Layer

Architecture Summary:
The MERN Node.js backend connects **directly to the OPERA Oracle database** via a hardened, encrypted tunnel — either an **IPSec site-to-site VPN** between the hotel network and the cloud provider, or an **SSL-wrapped TCP connection** over the hotel's static IP. A dedicated Oracle DB user is provisioned with tightly scoped `SELECT`, `INSERT`, and `UPDATE` permissions on reservation and booking tables only. The Node.js backend uses an Oracle client library (e.g., `node-oracledb`) to execute parameterised queries, with a thin ORM layer enforcing the permitted operation

set. MongoDB Atlas serves as the read replica, populated by a Change Data Capture (CDC) process on the Oracle side.

```
[MERN React Frontend]
        │
        ▼
[Node.js Express Backend]
        │ node-oracledb — TLS 1.3 over IPSec VPN / Static IP
        ▼
[Hotel Network — Static IP Endpoint]
        │
        ▼
[OPERA Oracle DB]
   ├── READ: SELECT on reservations, availability, rates
   └── WRITE: INSERT/UPDATE on RESERVATION, RESERVATION_NAME tables
        │
        ▼ (CDC / LogMiner / GoldenGate)
[MongoDB Atlas Read Replica]
        │
        ▼
[Stakeholder Dashboard]
```

Sync Mechanism: Oracle LogMiner or GoldenGate CDC streams change events from the OPERA Oracle DB to MongoDB Atlas in near real-time. Writes from the MERN backend go direct to Oracle and are immediately reflected in subsequent reads. This achieves the continuous sync required by TR-2 without polling latency.

Compliance Mapping:

- ☑ TR-1: Bidirectional direct DB connection; cloud initiates both reads and writes
- ☑ TR-2: CDC provides sub-second propagation of OPERA changes to Atlas; direct writes are immediately consistent
- ☑ TR-3: Static IP used for VPN endpoint or Oracle listener whitelisting
- ☑ SC-1: IPSec VPN + Oracle native encryption (TLS 1.3 / AES-256) on all DB connections
- ☑ SC-2: PII controlled at application layer; booking fields transmitted as required
- ☑ SC-3: Dedicated Oracle user with `SELECT`, `INSERT`, `UPDATE` on scoped tables; no `DELETE` or DDL
- ☑ SA-1/SA-2: MFA at MERN layer; stakeholder reads served from Atlas replica

Tech Stack: node-oracledb, Oracle LogMiner / GoldenGate, MongoDB Atlas, IPSec VPN, React
Complexity: ★★★★ Medium-High
Cost Profile: Medium (VPN infrastructure + GoldenGate licensing if used)

Pros:

- Direct DB access gives the MERN backend full control over query design, transaction management, and write operations — no API layer constraints limiting what can be expressed
- LogMiner-based CDC is the most reliable continuous sync mechanism available for Oracle — changes are captured at the redo log level, meaning nothing is missed even if the application layer is temporarily down
- The lowest latency read path of any solution in this document — the MERN backend can query Oracle directly with sub-millisecond round-trip on the VPN
- Static IP + IPSec VPN is a well-understood, auditable network security control that hotel IT teams and Oracle DBAs are familiar with

- No third-party SaaS dependency in the data path — the integration is entirely within your infrastructure control

Cons:

- Direct Oracle DB credentials in the cloud environment represent the highest security risk in this set; a compromised MERN backend has a direct write path to the OPERA production database
- Requires an Oracle DBA to provision and maintain the scoped DB user, manage connection pool sizing, and monitor for OPERA schema changes that could break queries
- Oracle GoldenGate (the production-grade CDC option) is a licensed Oracle product with significant cost; LogMiner is free but more operationally complex to maintain at scale
- IPSec VPN configuration between the hotel network and cloud provider requires coordination between hotel IT, the cloud provider, and potentially the property management company
- Any OPERA upgrade that alters the reservation table schema can silently break MERN queries or writes — requires a schema-change monitoring process

---

Solution 3: Node.js Sync Agent (On-Prem) + WebSocket Push + MongoDB Atlas

Architecture Summary:
A **Node.js sync agent** runs on-premises within the hotel network, connecting to both the OPERA Oracle DB locally and the MERN cloud service via a **persistent WebSocket or Server-Sent Events (SSE) connection**. Because the agent initiates the WebSocket connection outbound, no port forwarding is required — though a static IP is available if needed for connection stability. The agent listens to Oracle change events (via polling or OXI triggers), serialises the payload, and **pushes updates to the MERN backend in real-time**. For write operations, the MERN backend sends reservation creation/update commands back through the same WebSocket channel, and the agent applies them directly to the Oracle DB using scoped credentials.

```
[OPERA Oracle DB]◄──INSERT/UPDATE──[On-Prem Node.js Agent]
        │                                   ▲│
        │ Poll / OXI trigger                 │└ WebSocket (persistent, outbound)
        ▼                                   │▼
[Change Detected]                   [MERN Node.js Backend]
        │                                   │
        └──── serialise + push ────────────►│
                                            │
                                    [MongoDB Atlas]
                                            │
                                    [React Frontend]
                                            │
                                    [Stakeholder Dashboard]
```

Sync Mechanism: The WebSocket connection is full-duplex — OPERA changes flow up to the MERN backend continuously (satisfying TR-2), and reservation write commands flow down to the agent for immediate local execution. The agent acknowledges writes with a confirmation message, giving the MERN backend transactional feedback.

Compliance Mapping:

- ☑ TR-1: Agent initiates outbound WebSocket; MERN backend can push commands over the established channel
- ☑ TR-2: Real-time bidirectional push via persistent WebSocket; no polling lag for state changes

- ☑ TR-3: Static IP available but not required; WebSocket is outbound from the agent
- ☑ SC-1: WSS (WebSocket Secure) enforces TLS 1.3; Oracle connection uses native encryption
- ☑ SC-2: PII included only in reservation payloads where required
- ☑ SC-3: Agent uses scoped Oracle credentials: `SELECT` for reads, `INSERT`/`UPDATE` on reservation tables for writes
- ☑ SA-1/SA-2: MFA at MERN auth layer; Atlas serves stakeholder reads

Tech Stack: Node.js (on-prem agent + MERN backend), WebSocket/ws library, Oracle node-oracledb, MongoDB Atlas, React
Complexity: ★★★ Medium
Cost Profile: Low-Medium (no additional licensing; compute cost for agent process)

Pros:

- The on-prem agent is written in Node.js — the same language as the MERN backend — meaning the same development team can own, maintain, and deploy both ends of the integration without context switching
- Full-duplex WebSocket provides genuine real-time bidirectional sync, satisfying TR-2 without the complexity of separate read and write pipelines
- No Oracle credentials leave the hotel network — the agent holds DB credentials locally; the MERN backend only sends command payloads, never touches the DB directly
- Static IP is available but the architecture does not depend on it — if the hotel's IP changes, the agent reconnects outbound automatically
- Lightweight and low-cost — the agent is a Node.js process with no additional licensing requirements beyond the existing Oracle client library

Cons:

- A persistent WebSocket connection between the hotel and cloud must be kept alive across network interruptions, hotel Wi-Fi instability, and server restarts — requires robust reconnection logic, heartbeat monitoring, and a message replay buffer for missed events
- The on-prem Node.js agent is a custom-built component that must be versioned, deployed, and monitored at each hotel property — multi-property rollouts require a remote management strategy
- Write commands flowing from the MERN backend through the WebSocket to the local Oracle DB introduce a two-hop write latency (MERN → WebSocket → Agent → Oracle) compared to a direct DB connection
- If the WebSocket connection drops during a write operation, the MERN backend may not receive the acknowledgement — idempotency keys and retry logic are essential to prevent duplicate bookings
- The agent process represents a local dependency; a hotel IT team rebooting the server without restarting the agent would silently break sync until noticed

---

Solution 4: GraphQL Subscriptions + OPERA Web Services (OWS) + MongoDB Atlas

Architecture Summary:
Targeting on-premises OPERA installations that do not have OHIP available, this solution uses **OPERA Web Services (OWS)** — Oracle's SOAP/XML API for on-prem OPERA — as the write channel, fronted by a **GraphQL API layer** in the MERN backend. The MERN Node.js server exposes a GraphQL schema that maps directly to OPERA reservation operations. For reads, a **scheduled OWS polling service** hydrates MongoDB Atlas at high frequency (every 30–60 seconds). For writes, GraphQL mutations trigger synchronous OWS SOAP calls to OPERA, with the response used to confirm the booking before updating Atlas. The hotel's static IP is used to restrict OWS access to the cloud service exclusively.

```
[React Frontend]
        │ GraphQL query / mutation / subscription
        ▼
[Node.js GraphQL Server (Apollo)]
        ├── Query ──────────────────────────────► [MongoDB Atlas]
        │                                                ▲
        │                                                │ OWS Sync (30s poll)
        └── Mutation ──► [OWS SOAP Client]──────────────┤
                                  │                     │
                                  ▼                     │
                     [OPERA Web Services]               │
                          (Static IP)                   │
                                  │                     │
                                  ▼                     │
                     [OPERA Oracle DB]──────────────────┘
                      INSERT/UPDATE on
                      reservation tables
```

Sync Mechanism: Reads are served from Atlas (low latency, no OPERA load). Writes go synchronously through OWS, ensuring OPERA is the system of record. OWS polling every 30–60 seconds keeps Atlas consistent with walk-ins and PMS-originated changes. GraphQL subscriptions (via Atlas Change Streams) push real-time updates to connected React clients.

Compliance Mapping:

- ☑ TR-1: Bidirectional — cloud calls OWS for writes; high-frequency polling for reads
- ☑ TR-2: 30–60 second OWS sync cycle approaches near-real-time; writes are immediately consistent
- ☑ TR-3: Static IP used to whitelist cloud service IP on the hotel firewall for OWS port
- ☑ SC-1: OWS over HTTPS (TLS 1.3); Atlas TLS 1.3 in transit
- ☑ SC-2: PII handled per booking requirements
- ☑ SC-3: OWS credentials scoped to reservation operations; no direct Oracle access from cloud
- ☑ SA-1/SA-2: MFA at GraphQL auth layer; Atlas for stakeholder reads

Tech Stack: Apollo GraphQL Server, OPERA Web Services (SOAP), node-soap, MongoDB Atlas, React, Atlas Change Streams
Complexity: ★★★ Medium
Cost Profile: Low-Medium (OWS is included with OPERA on-prem license; Atlas M10+)

Pros:

- OWS is available on on-premises OPERA installations without an OHIP subscription, making this the most accessible write channel for hotels not yet on OPERA Cloud
- GraphQL's typed schema provides a clean, self-documenting contract between the React frontend and the backend — mutations for `createReservation`, `updateReservation` are explicit and versioned
- GraphQL subscriptions combined with Atlas Change Streams give the React frontend a real-time push notification layer with no additional infrastructure
- OWS has been the standard OPERA integration mechanism for on-prem deployments for many years — it is well-documented and Oracle-support-approved, reducing implementation risk
- Atlas absorbing all read traffic means the OPERA server is only hit for writes and sync polling, significantly reducing load on the production PMS

Cons:

- OWS uses SOAP/XML, which is verbose, slower to parse, and significantly less developer-friendly than REST or GraphQL — the Node.js `node-soap` library adds complexity and the XML response mapping requires careful maintenance
- The 30–60 second polling cycle means Atlas can be up to a minute behind OPERA for PMS-originated changes — this may be acceptable for most reads but could cause brief availability discrepancies visible to booking users
- Static IP requirement on the hotel firewall means coordinating a port-open rule with hotel IT for the OWS HTTP port — this is manageable but requires change-control processes at each property
- OWS authentication uses a WSSE username/token pattern that requires careful credential rotation management in the MERN backend
- OWS feature coverage varies by OPERA version; advanced reservation features (packages, linked profiles, allotments) may require multiple SOAP call chains to execute a single logical operation

---

Solution 5: Event-Driven Sync via Oracle GoldenGate / Debezium CDC + MERN Event Bus + OHIP Writes

Architecture Summary:
The most robust and enterprise-grade option. **Change Data Capture (CDC)** via Oracle GoldenGate (or open-source Debezium with Oracle LogMiner) continuously streams every committed change in the OPERA Oracle DB — at the redo log level — to a **Kafka or AWS MSK event bus** in the cloud. The MERN Node.js backend consumes these events and applies them to MongoDB Atlas in milliseconds, achieving true continuous sync per TR-2 with no polling whatsoever. Write operations from the MERN backend flow through **OHIP REST API** (for OPERA Cloud) or **OWS** (for on-prem OPERA), providing a clean, Oracle-validated write channel. The hotel's static IP is used to secure the GoldenGate replication endpoint.

```
[OPERA Oracle DB]
        │ Redo log capture
        ▼
[GoldenGate / Debezium CDC Agent]
        │ TLS 1.3 — Static IP outbound
        ▼
[Kafka / AWS MSK Event Bus]
        │
        ├──► [MERN Node.js Consumer] ──► [MongoDB Atlas]
        │                                        │
        │                              [React Frontend]
        │                              [Stakeholder Dashboard]
        │
        └──► [Audit / Analytics Pipeline]

[MERN Write Path]
[React Frontend] ──► [Node.js Backend] ──► [OHIP / OWS] ──► [OPERA Oracle DB]
                                                  │
                                        (Confirmation event
                                         re-enters Kafka via CDC)
```

Sync Mechanism: CDC is the gold standard for continuous sync — every INSERT, UPDATE, and committed transaction in OPERA is captured at the database engine level and propagated to Kafka within milliseconds. Atlas is kept perpetually current. Writes from the MERN backend go through OHIP/OWS and, once committed to Oracle, are automatically re-captured by CDC and reflected in Atlas — creating a closed, self-consistent loop.

Compliance Mapping:

- ☑ TR-1: Bidirectional — CDC pushes OPERA changes outbound; OHIP/OWS handles cloud-to-OPERA writes
- ☑ TR-2: CDC provides millisecond-level continuous sync — the strongest compliance with TR-2 of any solution
- ☑ TR-3: Static IP used to secure GoldenGate replication target endpoint; no raw DB port exposed
- ☑ SC-1: GoldenGate TLS 1.3; Kafka TLS 1.3; OHIP TLS 1.3; Atlas TLS 1.3 — full chain enforced
- ☑ SC-2: PII fields can be masked in the CDC transformation layer before events reach Kafka
- ☑ SC-3: CDC agent uses a dedicated Oracle log-mining user; OHIP/OWS handles writes without direct DDL access
- ☑ SA-1/SA-2: MFA at MERN auth layer; Atlas and Kafka ACLs restrict consumer access

Tech Stack: Oracle GoldenGate or Debezium, Apache Kafka / AWS MSK, OHIP or OWS, Node.js, MongoDB Atlas, React
Complexity: ★★★★★ High
Cost Profile: High (GoldenGate licensing + Kafka cluster + OHIP if applicable)

Pros:

- The only solution that achieves genuinely continuous, sub-second sync from OPERA to Atlas without any polling — CDC at the redo log level captures every committed transaction, including PMS-originated changes, walk-ins, and back-office edits that no API would surface
- Kafka as an event bus decouples OPERA from the MERN backend entirely — the PMS does not need to know the SaaS exists; all integration logic lives in consumers
- Event replay: if the MERN backend or Atlas has an outage, Kafka retains the event log and consumers can replay from the last committed offset — zero data loss
- The write path (OHIP/OWS) and read path (CDC) are completely independent — a write API outage does not affect read sync, and vice versa
- Highly extensible: additional consumers can be added to the Kafka bus for analytics, reporting, revenue management, or third-party integrations without modifying OPERA or the MERN backend

Cons:

- The most expensive and operationally complex solution in this document; Oracle GoldenGate licensing is enterprise-priced and the operational expertise to run it is scarce
- Debezium with Oracle LogMiner is the open-source alternative but requires careful tuning — LogMiner has known performance implications on high-transaction Oracle databases and can impact OPERA server load if not configured correctly
- Kafka requires its own deployment, monitoring, topic management, consumer group administration, and retention policy — a non-trivial operational commitment on top of the existing MERN stack
- The combination of CDC + Kafka + OHIP + Atlas is the largest surface area in this document; debugging a sync discrepancy requires tracing an event across four distinct systems
- Initial setup involves significant Oracle DBA involvement to configure supplemental logging, GoldenGate extract processes, and trail files — this is not a self-service deployment

---

3. Comparison Matrix

Solution	Write Channel	Read Channel	Sync Model	Real-Time?	Static IP Required	Complexity	Cost

Solution	Write Channel	Read Channel	Sync Model	Real-Time?	Static IP Required	Complexity	Cost
1. OHIP Bidirectional	OHIP REST API	Atlas + OHIP	Push (webhooks) + Pull	☑ Near real-time	Optional	★★★	Med-High
2. Direct Oracle + VPN	node-oracledb direct	Atlas CDC	CDC (LogMiner)	☑ Sub-second	☑ Required	★★★★	Medium
3. On-Prem Node.js Agent + WebSocket	Agent → Oracle (local)	Atlas	Full-duplex WebSocket	☑ Real-time	Optional	★★★	Low-Med
4. GraphQL + OWS	OWS SOAP	Atlas	Poll (30–60s) + OWS	⚠ Near (30–60s lag)	☑ Recommended	★★★	Low-Med
5. CDC + Kafka + OHIP/OWS	OHIP or OWS	Atlas via Kafka	CDC (redo log)	☑ Milliseconds	☑ Recommended	★★★★★	High

Write Capability by Table

Solution	`RESERVATION`	`RESERVATION_NAME`	`ALLOTMENT`	`RATE_HEADER`	Direct DDL
1. OHIP	☑ OHIP-validated	☑ OHIP-validated	⚠ Limited by OHIP endpoints	✕	✕
2. Direct Oracle	☑ Full control	☑ Full control	☑ Full control	⚠ Requires explicit grant	✕
3. WebSocket Agent	☑ Agent-mediated	☑ Agent-mediated	☑ Agent-mediated	⚠ Configurable	✕
4. OWS	☑ OWS-validated	☑ OWS-validated	⚠ Limited by OWS methods	✕	✕
5. CDC + OHIP/OWS	☑ OHIP or OWS	☑ OHIP or OWS	⚠ Depends on write channel	✕	✕

4. Recommended Architecture

Primary Recommendation: Solution 3 (On-Prem Node.js Agent + WebSocket)

For a MERN-based team building a new SaaS product, **Solution 3** offers the best balance of real-time sync performance, write capability, and operational simplicity:

- The agent is written in the same language as the rest of the stack — no Oracle DBA required, no SOAP libraries, no separate licensing
- Real-time bidirectional WebSocket satisfies TR-2 continuously without polling lag

- Oracle credentials never leave the hotel network — the cloud service is insulated from direct DB access
- Static IP is available if needed but the architecture does not depend on it, making multi-property rollout straightforward

Secondary Recommendation: Solution 1 (OHIP Bidirectional)

If the hotel group is on **OPERA Cloud** and the budget supports OHIP licensing, Solution 1 is the most architecturally clean option. OHIP handles all Oracle complexity, write validation is Oracle-enforced, and the MERN backend never needs an Oracle client library.

For Maximum Sync Fidelity: Solution 5 (CDC + Kafka)

If the non-negotiable requirement is **zero tolerance for sync lag** across many properties at scale, Solution 5 is the only architecture that delivers millisecond-level continuous sync. The cost and complexity are justified only at enterprise scale (10+ properties) or where financial transactions depend on real-time availability data.

What to Avoid

Solution 2 (Direct Oracle + VPN) should be treated as a last resort. While it offers maximum query flexibility, placing direct Oracle write credentials in the cloud environment creates unacceptable risk to the OPERA production database. If chosen, it must be paired with a Web Application Firewall, connection proxying, and a rigorous credential rotation policy.

Document Version 2.0 — prepared for review. All solutions subject to Oracle OPERA PMS version compatibility assessment and hotel IT infrastructure audit prior to implementation. SC-3 write permissions require sign-off from the property's Oracle DBA and IT Security team before provisioning.